

TP3 — Développer un script modulaire interrogeant une API réseau

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description

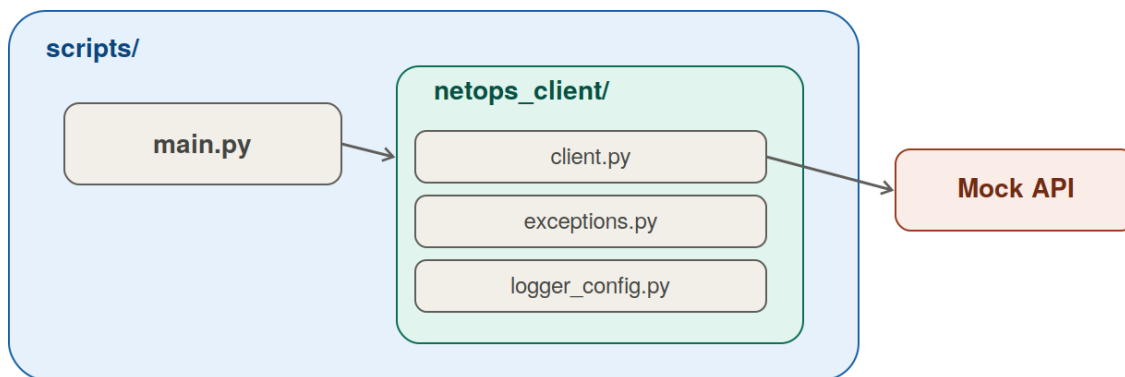


Figure 1: Structure du package `netops_client`

Ce TP construit un package Python modulaire (exceptions, configuration des logs, client métier, point d'entrée) qui interroge une API réseau REST et retourne l'état des interfaces d'un équipement. Faute de matériel Cisco disponible sur les postes, l'API est simulée par un petit serveur Flask (`mock_api.py`) qui répond comme le ferait un vrai équipement. Ce package sera réutilisé et enrichi tout au long de la formation (DNA Center, ACI, SD-WAN utilisent tous des APIs REST similaires).

Prérequis

- TP1 et TP2 validés
- Python 3.9+ installé et accessible (`python --version`)
- `pip` accessible (voir TP2 si besoin de corriger le PATH)

Étapes

Étape 1 — Créer sa branche pour ce TP

```
cd netops-project-<nom-groupe>
git checkout dev
git pull origin dev
git checkout -b <prenom>-<nom>-jour1-tp3
```

Étape 2 — Installer les dépendances

```
pip install requests flask
```

Étape 3 — Créer le mock d'API réseau (simule l'équipement)

```
cat > scripts/mock_api.py << 'EOF'
from flask import Flask, jsonify

app = Flask(__name__)

INTERFACES = {
    "GigabitEthernet0/1": {"status": "up", "ip": "10.0.1.1", "vlan": 10},
    "GigabitEthernet0/2": {"status": "down", "ip": None, "vlan": 20},
    "GigabitEthernet0/3": {"status": "up", "ip": "10.0.3.1", "vlan": 30},
}

@app.route("/api/devices/<device_id>/interfaces", methods=["GET"])
def get_interfaces(device_id):
    return jsonify({"device_id": device_id, "interfaces": INTERFACES})

if __name__ == "__main__":
    app.run(port=5000, debug=False)
EOF
```

Lancez-le dans un terminal dédié, laissé ouvert :

```
python scripts/mock_api.py
```

Ouvrez un **second terminal** (cd netops-project-<nom-groupe>) pour la suite.

Si Flask refuse de démarrer (An attempt was made to access a socket in a way forbidden by its access permissions), un autre programme occupe déjà le port 5000. Identifiez-le :

```
netstat -ano | grep ":5000"
tasklist //FI "PID eq <numero_pid_trouve>"
```

Sur beaucoup de PC Windows, ce port est occupé par une appli déjà lancée (VPN, Docker Desktop, service de découverte réseau...). Fermez l'appli concernée, ou changez de port :

```
sed -i 's/app.run(port=5000, debug=False)/app.run(port=8000, debug=False)/' sc
# et adapter base_url="http://localhost:8000" a l'etape 5 en consequence
```

Étape 4 — Créer le package modulaire netops_client

```
mkdir -p scripts/netops_client
```

exceptions.py — le module des erreurs personnalisées :

```
cat > scripts/netops_client/exceptions.py << 'EOF'
class NetworkAPIError(Exception):
    """Erreur levee lorsque l'API reseau repond en erreur ou est injoignable."""
    pass
EOF
```

logger_config.py — le module de configuration des logs, réutilisable par tout le package :

```
cat > scripts/netops_client/logger_config.py << 'EOF'
import logging

def setup_logger(name, log_file="scripts/logs_network_client.log"):
    """Cree et configure un logger reutilisable par tous les modules du package."""
    logger = logging.getLogger(name)
    if logger.handlers:
        return logger

    logger.setLevel(logging.INFO)
    formatter = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s")

    file_handler = logging.FileHandler(log_file)
    file_handler.setFormatter(formatter)

    console_handler = logging.StreamHandler()
    console_handler.setFormatter(formatter)

    logger.addHandler(file_handler)
    logger.addHandler(console_handler)
    return logger
EOF
```

client.py — le module métier, qui importe les deux précédents :

```
cat > scripts/netops_client/client.py << 'EOF'
import requests

from .exceptions import NetworkAPIError
from .logger_config import setup_logger

logger = setup_logger(__name__)
```

```

class NetworkClient:
    """Client modulaire pour interroger une API reseau REST."""

    def __init__(self, base_url, timeout=5):
        self.base_url = base_url.rstrip("/")
        self.timeout = timeout

    def get_interfaces(self, device_id):
        url = f"{self.base_url}/api/devices/{device_id}/interfaces"
        logger.info(f"Requete GET vers {url}")
        try:
            response = requests.get(url, timeout=self.timeout)
            response.raise_for_status()
        except requests.exceptions.RequestException as e:
            logger.error(f"Echec de la requete API : {e}")
            raise NetworkAPIError(f"Impossible de contacter l'API : {e}")

        data = response.json()
        logger.info(f"Reponse recue pour {device_id} : {len(data['interfaces'])} interfaces")
        return data["interfaces"]

    def count_interfaces_down(self, device_id):
        interfaces = self.get_interfaces(device_id)
        down = [name for name, info in interfaces.items() if info["status"] == "down"]
        if down:
            logger.warning(f"Interfaces down sur {device_id} : {down}")
        return down
EOF

```

__init__.py — expose le package proprement (from netops_client import NetworkClient):

```

cat > scripts/netops_client/__init__.py << 'EOF'
from .client import NetworkClient
from .exceptions import NetworkAPIError

__all__ = ["NetworkClient", "NetworkAPIError"]
EOF

```

Étape 5 — Créer le point d'entrée main.py

```

cat > scripts/main.py << 'EOF'
from netops_client import NetworkClient, NetworkAPIError

if __name__ == "__main__":
    client = NetworkClient(base_url="http://localhost:5000")

```

```

try:
    interfaces = client.get_interfaces("router-siege-01")
    for name, info in interfaces.items():
        print(f"{name} : {info['status']} (ip={info['ip']}, vlan={info['vlan']})

    down = client.count_interfaces_down("router-siege-01")
    print(f"\nInterfaces en panne : {down if down else 'aucune'}")

except NetworkAPIError as e:
    print(f"Erreur reseau : {e}")
EOF

```

Étape 6 — Vérifier la structure

```
find scripts -type f | sort
```

Vous devez voir :

```

scripts/main.py
scripts/mock_api.py
scripts/netops_client/__init__.py
scripts/netops_client/client.py
scripts/netops_client/exceptions.py
scripts/netops_client/logger_config.py

```

Étape 7 — Tester

Important : lancez depuis la racine du projet, pas depuis scripts/ (le chemin du fichier de log en dépend) :

```

cd ~/Desktop/netops-project
python scripts/main.py

```

Résultat attendu :

```

GigabitEthernet0/1 : up (ip=10.0.1.1, vlan=10)
GigabitEthernet0/2 : down (ip=None, vlan=20)
GigabitEthernet0/3 : up (ip=10.0.3.1, vlan=30)

```

```
Interfaces en panne : ['GigabitEthernet0/2']
```

Un fichier scripts/logs_network_client.log doit être créé, avec une ligne par requête/événement.

Étape 8 — Tester la gestion d'erreur

Arrêtez le mock (Ctrl+C dans son terminal), puis relancez :

```
python scripts/main.py
```

Le script doit afficher une erreur claire (Erreur reseau : Impossible de contacter l'API : ...), pas un traceback Python brut, et logger l'erreur en ERROR dans le fichier de log. Relancez ensuite le mock avant de continuer.

Étape 9 — Committer et pousser

```
git add scripts/  
git commit -m "TP3 : package modulaire netops_client (exceptions, logger, client, r  
git push -u origin <prenom>-<nom>-jour1-tp3
```

Ouvrir une pull request <prenom>-<nom>-jour1-tp3 → dev.

Comprendre le TP : l'API, le mock, et les logs

mock_api.py joue le rôle de l'équipement réseau : un petit serveur Flask qui écoute sur http://localhost:5000 et répond, en JSON, à toute requête GET /api/devices/<id>/interfaces — exactement comme le ferait un vrai routeur exposant une API REST. main.py et le package netops_client jouent le rôle de l'outil d'automatisation : un script qui envoie de vraies requêtes HTTP à cette API pour lire l'état des interfaces, au lieu de s'y connecter manuellement en SSH.

Chaque ligne du fichier de log correspond à une action réelle du client : une ligne INFO avant chaque requête GET, une ligne INFO quand la réponse est reçue et décodée, et une ligne WARNING dès qu'une interface down est détectée (une anomalie mérite plus d'attention qu'un événement normal). Le préfixe netops_client.client indique quel module a écrit la ligne — utile dès que le projet aura plusieurs modules qui loggent en parallèle. Quand le mock est arrêté, requests lève une exception réseau ; le code l'attrape, la logge en ERROR, puis la retransforme en NetworkAPIError, une erreur métier claire que main.py sait afficher proprement plutôt que de laisser Python planter avec un message technique illisible.

Validation

- ☐ Le mock API démarre sur http://localhost:5000 (ou un autre port si celui-ci était occupé)
- ☐ La structure du package netops_client (exceptions, logger_config, client, __init__) est en place
- ☐ python scripts/main.py (lancé depuis la racine du projet) affiche les 3 interfaces avec leur statut
- ☐ Le script détecte et affiche correctement l'interface en panne (GigabitEthernet0/2)
- ☐ Un fichier scripts/logs_network_client.log est généré et contient les logs, avec le nom du module (netops_client.client)
- ☐ Quand le mock est arrêté, le script affiche une erreur claire (pas de traceback brut) et logge l'erreur en ERROR
- ☐ Une pull request <prenom>-<nom>-jour1-tp3 → dev est ouverte